Unit: a small dependent type checker with segments as modules

17 October 2026

Abstract

Unit is a prototype proof checker for a dependent type theory with Π, Σ, records, labelled sums, an identity type with its eliminator, and a hierarchy of cumulative universes indexed by level expressions. Its module system is based on the segments of Automath, in the form given in [4]: a module is a list of declarations, its parameters are the declarations without a body, and an instance only binds the parameters, without copying the body. The same segments also serve as record types. The prototype has been used to formalise a model of univalent type theory over posets and simplicial sets, about 200 files. This note describes the type theory, the module system, and the choices made to keep checking predictable and fast. The name Unit refers to the units, the declarations of which every file, segment and local block is a list.

Contents

1 Overview **1**

2 The type theory **2**
2.1 Terms . . . 2
2.2 Universes and levels . . . 3
2.3 Definitional equality . . . 3

3 The module system **4**
3.1 Segments . . . 4
3.2 Instances share . . . 5
3.3 Parameters stated once . . . 5
3.4 Segments as types . . . 6

4 The checker **6**

5 Notation read before checking **7**
5.1 Omitted arguments . . . 7
5.2 Equational chains . . . 7
5.3 Scoping . . . 7

1 Overview

A file is a list of *units*. A unit is a declaration of one of the following kinds:

`x (tel) : A ;`	a parameter (at the top level: a primitive notion)
`x (tel) : A = e ;`	a definition
`data D (tel) = c1 (tel1) | ...;`	a labelled sum (a definition)
`level a b ;`	level variables
`constraint a < b ;`	constraints between levels
`module M (tel) { units } ;`	a segment
`module N = M[args] ;`	an instance of a segment
`open M ; import F ;`	scoping
`opaque f ; transparent f ;`	the unfolding strategy

Checking a list of units threads a *context of units*: each unit is checked in the context of the units before it, and is then added to it. A definition stays a definition in the context: its body is available for computation. The bodies of `let` and `where` are lists of units as well. Consecutive definitions form one mutually recursive group.

Two design principles run through the system. First, *nothing is guessed*: there is no unification, and the few arguments that may be omitted are found by a deterministic matching procedure (Section 5.1). Second, *nothing is copied*: a definition has one identity wherever it is used, and two uses are compared by that identity before anything is unfolded (Section 2.3).

2 The type theory

2.1 Terms

The terms are those of a dependent type theory with definitions:

`U l`	the universe at level `l`
`(x : A) -> B, \ x -> t, t u`	dependent functions
`(x : A) * B, (t, u), t.1, t.2`	dependent pairs
`sig { l1 : A1 ; ...}, struct { l1 = t1 ; ...}, t.l`	records
`c t1 ...tn, split { c1 x1...-> t1 ; ...}`	constructors and case analysis
`Id A a b, refl a, idJ A a C d b p`	the identity type
`[a] A, <a> t, t @l`	products over levels
`[a < b] A`	products over constraints
`let { units } in t, t where { units }`	local definitions
`M[args].x`	a member of a segment instance

Labelled sums. A `data` declaration introduces a type with named constructors, each with a telescope of arguments; the declaration may be recursive. A function out of a sum is written `split`, a list of branches, one per constructor; `split` is not a term on its own but is checked against a function type whose domain is a sum, as in

```
data N = zero | suc (n : N) ;
plus : N -> N -> N = split {
  zero  -> \ n -> n ;
  suc p -> \ n -> suc (plus p n) } ;
```

Recursion is through the name of the definition. There are no indexed inductive families. Neither termination nor positivity is checked.

The identity type. `Id A a b` is a primitive type with `refl a : Id A a a` and the eliminator

$$\frac{C : (y : A) \to \mathsf{Id}\, A\, a\, y \to \mathsf{U}\, l \qquad d : C\, a\, (\mathsf{refl}\, a) \qquad p : \mathsf{Id}\, A\, a\, b}{\mathsf{idJ}\, A\, a\, C\, d\, b\, p : C\, b\, p}$$

with the computation rule $\mathsf{idJ}\, A\, a\, C\, d\, a\, (\mathsf{refl}\, a) = d$. Since there is no pattern matching on `refl`, there is no way to derive uniqueness of identity proofs: the theory is compatible with univalence, which a development may assume as a parameter (Section 3.3).

2.2 Universes and levels

The universes follow the system of [2]. Levels are expressions

$$l \;::=\; a \;\mid\; l^+ \;\mid\; l \vee m$$

in level variables a, a successor and a maximum. There is no level 0 for the user. Level variables are declared by `level a`, and may be constrained by `constraint` units ($l < m$, $l \leq m$, $l = m$).

The universes are *à la Russell*: an element of `U l` is itself a type. The rules are

- $\mathsf{U}\, l : \mathsf{U}\, l^+$;
- Π, Σ, records and Id lie in the maximum of the levels of their components;
- cumulativity: if $A : \mathsf{U}\, l$ and $l \leq m$ then $A : \mathsf{U}\, m$. Cumulativity applies to the whole type and not under a binder: a function into $\mathsf{U}\, l$ is not silently a function into $\mathsf{U}\, m$, and one writes $\lambda b.\, T\, b$ when needed;
- a sum declared without a universe, like `N` above, is *small*: it lies in every universe. A sum may also be declared at a given level, `data D : U l = ...`.

Products over levels and constraints. A type may be polymorphic in a level: `[a] A` is the type of the `<a> t`, applied to a level by `t @l`. A type may also depend on a constraint: `[a < b] A`. A constraint product is introduced and eliminated silently: a term of type $[\psi]A$ is used as a term of type A wherever ψ holds in the current context. Both kinds of products are *large*: they lie in no universe.

Deciding constraints. Whether a constraint follows from those of the context is decided by the algorithm of Bezem and Coquand for join-semilattices with an inflationary endomorphism: constraints become Horn clauses, and the question is answered by computing a least model by forward reasoning [1]. The same algorithm detects *loops*, contexts whose constraints force $l < l$. A context with a loop is inconsistent, and in it every two terms are convertible. In practice this signals an error in the constraints, and the checker warns.

2.3 Definitional equality

Definitional equality is generated by

- β for functions, pairs, records, level abstractions and `split` applied to a constructor;
- the computation rule of `idJ` on `refl`;
- the unfolding of definitions (but see `opaque` below);
- η for functions, pairs, records and elements of segment types.

The checker decides it as follows. Two values are first compared *structurally*, without any computation. A use of a definition is represented by its name together with the values of the variables its body depends on (its *arguments*, computed once, when the definition is declared); two uses of the same definition with equal arguments are equal without unfolding. Only when this fails are both sides brought to head normal form, unfolding definitions, and compared component by component.

Records are compared by name. A record type `sig{...}` and a record element `struct{...}` are compared by their occurrence in the text: two occurrences of the same `sig` are equal when the variables they depend on have equal values, and two different occurrences are different types even if their fields agree. This is the rule of the proof checker HALF [3] and of Gofer, and it avoids comparing large records field by field. In practice a record comes with one named constructor (Section 3.4). Dependent pairs $(x : A) * B$ are compared structurally.

The unfolding strategy. `opaque f` declares that, from this point on, `f` is not unfolded: it behaves as a constant, though its body has not changed. `transparent f` undoes this. The strategy is a property of the checking run and not of the values: a value is computed in the same way whatever the strategy, and only the comparison consults it. In particular an `opaque` declaration placed after a segment applies to every instance of it. Making a definition opaque can only make fewer terms convertible, so it never makes an incorrect proof correct. The intended use is to lock a construction right after the lemmas that characterise it, as one does in informal mathematics.

3 The module system

3.1 Segments

A segment is a named list of units:

```
module Mon (A : U a) (e : A) (op : A -> A -> A) {
  twice (x : A) : A = op x x ;
  four (x : A) : A = twice (twice x) ;
} ;
```

The *parameters* of a segment are its units without a body: the telescope in the header and the parameters declared in the body. The level variables and the constraints of its body belong to it as well. A segment is checked once, where it is declared, with its parameters as unknowns.

An *instance* `Mon[N, zero, plus]` gives values to the parameters, in order. Each argument is checked against the type of its parameter, in which the earlier arguments have been substituted; a level parameter takes a level; a constraint of the segment must hold for the given levels. Members are then accessed as `Mon[N, zero, plus].four`, or through an alias `module NM = Mon[N, zero, plus] ;` as `NM.four`, or after `open Mon[N, zero, plus] ;` as `four`. Arguments may also be given by name, `Mon[A := N, ...]`.

Segments nest. A segment declared inside another sees the parameters of the enclosing one lexically, and needs no arguments for them:

```
module Outer (A : U a) {
  module Inner { k (x : A) : A = x ; } ;
  kk (x : A) : A = Inner.k (Inner.k x) ;
} ;
```

`import F ;` reads the file `F.seg` and inserts it at that point as a segment `module F { ...} ;`, once. An import inside a segment therefore makes the file a sub-segment, which sees the enclosing parameters.

3.2 Instances share

An instance does not copy the body of the segment. It only records the values of the parameters, in an environment; the members are evaluated in that environment when they are used. A member therefore keeps a single identity, its qualified name, and two occurrences of a member are compared by this name and the values of the parameters it actually depends on. In particular `Mon[N, zero, plus].twice` reached through two different aliases or two different `open`s is one object, and the equation

```
share : Id (N -> N) (Mon[N, zero, plus].twice) (NM.twice) = refl NM.twice ;
```

holds without unfolding `twice`. When a large development instantiates the same segment many times, as a formalisation of a model typically does (a construction made over an abstract base, then applied to each instance), this is what keeps checking fast: equal things are recognised as equal by their names.

3.3 Parameters stated once

Because segments nest and share, assumptions of a whole development are stated once, as parameters of one outer segment. For instance the model formalised with Unit assumes function extensionality and univalence for the metatheory; the main file reads

```
module Model (fe : [b] Axioms[b].FunExt) (univ : [b] Axioms[b].Univalence) {
  import V ;
  import Presheaf ;
  ...
} ;
```

and every imported file sees `fe` and `univ`. Likewise the abstract base of the model (a category with an interval and a presheaf of cofibrations) is one segment whose parameters are the data of the base; the generic constructions are imported inside it, and an instance of the base (posets, or simplicial sets) is one instance of that segment, after which every generic construction is a member of the instance. Excluded middle, used at one place, is the parameter of a separate segment, so the files that may use it are exactly those imported inside that segment.

Changing the assumptions. Function extensionality follows from univalence. Once this is proved, as a segment `FunExtUA` producing, from univalence at a level b, function extensionality at b, the whole development is obtained under univalence alone by one instance:

```
module ModelUA (univ : [b] Axioms[b].Univalence) {
  feU : [b] Axioms[b].FunExt = <b> FunExtUA[b].FE[univ @b].fe ;
  module M = Model[feU, univ] ;
} ;
```

Every result of the development is then available as `ModelUA[univ].M.....` Since an instance only binds the parameters, checking it checks the two arguments and nothing of the body: the development is neither copied nor checked again.

3.4 Segments as types

A segment is also a *type*: the type of its instances, that is, the dependent record of its parameters. Its defined members are available on every element, as methods. With

```
module Mon (l : Level) {
  A : U l ;  op : A -> A -> A ;
  sq (x : A) : A = op x x ;
  e : A ;  sqe : Id A (sq e) e ;
} ;
```

- `Mon[a]` is a type (the level parameters must be given), and `new Mon[a, N, plus, zero, refl zero]` is an element;
- for `M : Mon[a]`, `M.op` is a field and `M.sq` is the member evaluated in `M`; `open M` gives them short names;
- a partial instance `Mon[a, N]` or `Mon[l := a, A := N]` is the type of the elements with those parameters fixed; for `M : Mon[a, N]`, `M.A` computes to `N`;
- fixing more parameters gives a subtype: `Mon[a, N]` is a subtype of `Mon[a]`;
- elements have η: `M` is equal to `new Mon[a, M.A, M.op, M.e, M.sqe]`;
- the universe of `Mon[a]` is the maximum of the universes of its parameters.

A segment whose body has fields comes with a constructor `mkMon`, defined right after it. Since elements are compared by occurrence, as records are, using this one constructor everywhere makes elements built from equal data equal. This gives records in the style of Arend's classes and of Half's theories, within the same mechanism as modules.

4 The checker

The checker is written in Haskell, about 4500 lines, with no library beyond the base. Its parts are: levels and the decision of constraints; evaluation into values with closures; head normalisation under the unfolding strategy; conversion; type checking of units; the filling of holes; and the parser and scope resolution, which also computes, for each definition, the variables it depends on.

The context after a unit does not depend on whether the unit checked: definitions enter the context by their types and bodies, not by their proofs of correctness. Hence the result of each declaration can be computed separately, with a time budget (ten seconds per declaration; an overrun counts as a failure), and one file can be checked while the others are trusted. The checker also runs as a server that loads a development once and, after an edit, checks only the files whose content changed and those depending on them; the 205 files of the model check from scratch in about 28 seconds.

Remark 1. What Unit does not have: no unification and no implicit arguments beyond the matching of Section 5.1; no indexed inductive families; no termination or positivity check; no pattern matching on `refl`; no proof irrelevance. These omissions are deliberate: each would make checking less predictable, and the formalisation did not need them.

5 Notation read before checking

The following conveniences are not part of the core language: they are translated into core terms before checking, and the core never sees them.

5.1 Omitted arguments

A parameter written in braces, `f {x : A} (y : B) : C`, is *hidden*: the type of `f` is the ordinary $(x : A) \to (y : B) \to C$, and a use `f u` is read as `f _ u`, where `_` is a hole. A hole may also be written explicitly anywhere, and a hidden argument may be given explicitly as `{e}`. Holes are filled before the term is checked, by one-way *matching*, never by unification:

- the inferred type of each explicit argument is matched against the corresponding domain, with the holes as unknowns; two applications are matched argument by argument only when they have the same head; otherwise both are computed and matching continues on their head normal forms;
- the expected type only fills holes left open by the explicit arguments;
- two different solutions for one hole are an error, so the result does not depend on the order;
- a solution is written back as a term in the current scope, using names only.

A hole that is not determined this way is an error that asks for an explicit `{e}`. This works well for lemmas about paths, such as `transH p (symH q)`, whose endpoints are determined by the explicit arguments, and it is deliberately not clever.

5.2 Equational chains

The notation

```
calc A { a0 =< p1 > a1 =< p2 > a2 ... =< pn > an }
```

proves $\mathsf{Id}\, A\, a_0\, a_n$. It stands for the right-nested composite of the paths p_i; each p_i is checked against $\mathsf{Id}\, A\, a_{i-1}\, a_i$, so every point is written once and an error is reported at its step.

5.3 Scoping

`open M ;` (or `open M[args] ;`) makes the members of a segment available by their short names. The resolver replaces every short name by the qualified name it stands for, `M[args].x`, so the core only sees qualified names. For a variable `c` of a segment type, `open c` gives short names to `c.x`. `import F ;` is expanded when the files are read: the file `F.seg` is inserted as `module F { ...} ;`, once. A `where` block is a `let`, and the constructor `mkM` of a segment type (Section 3.4) is an ordinary definition added by the resolver after the segment.

References

[1] M. Bezem and T. Coquand. Loop-checking and the uniform word problem for join-semilattices with an inflationary endomorphism. *Theoretical Computer Science* 913, 2022.

[2] M. Bezem, T. Coquand, P. Dybjer and M. Escardó. Type theory with explicit universe polymorphism. arXiv:2212.03284, 2022 (revised and extended version, 2024).

[3] J. Cederquist. *A Pointfree Approach to Constructive Analysis in Type Theory*. PhD thesis, Chalmers University of Technology and University of Göteborg, 1997. Section 7.1 describes the proof checker HALF.

[4] Q. Wang. *A Haskell Implementation for a Dependent Type Theory with Definitions*. Master's thesis, Chalmers University of Technology and University of Gothenburg, 2021.